Neural Networks

Libraries used:

- numpy
- pandas
- matplotlib
- seaborn
- scikit-learn
- imbalanced-learn
- tensorflow

Data Preprocessing

1. Handling Incorrect Data Types

- TotalCharges was converted to numeric field named TotalCharges_numeric using pd.to_numeric().
- Once conversion, missing values were checked there were 11 rows. This is expected because previously invalid string became NaN.

From the EDA we saw that total charge column was approximately monthly charges multiplied into tenure. So, the difference (estimated and the real value) for all the rows in monthly charges was calculated and the mean value of that (0.15) was added with the (monthly charges * tenure) value for the 11 missing values.

$$\text{TotalCharges} = (\text{Monthly Charges} * \text{Tenure}) + 0.15 \text{ (Mean value of difference)}$$

The missing count was very small, and removing rows would reduce data, which is not ideal for neural networks.

Then the original object type TotalCharges column was dropped.

2. Removing Irrelevant Columns

The column customerID was removed as it does not provide any information and acts only as an identifier.

3. Feature Selection Using Chi-Square Test

To identify the most influential categorical predictors:

- The Chi-Square test was conducted to all encoded categorical variables.
- The most prominent 12 columns with the leading Chi-Square scores were preferred.
- All remaining columns were dropped to reduce noise and dimensionality, ensuring the neural network trains effectively.

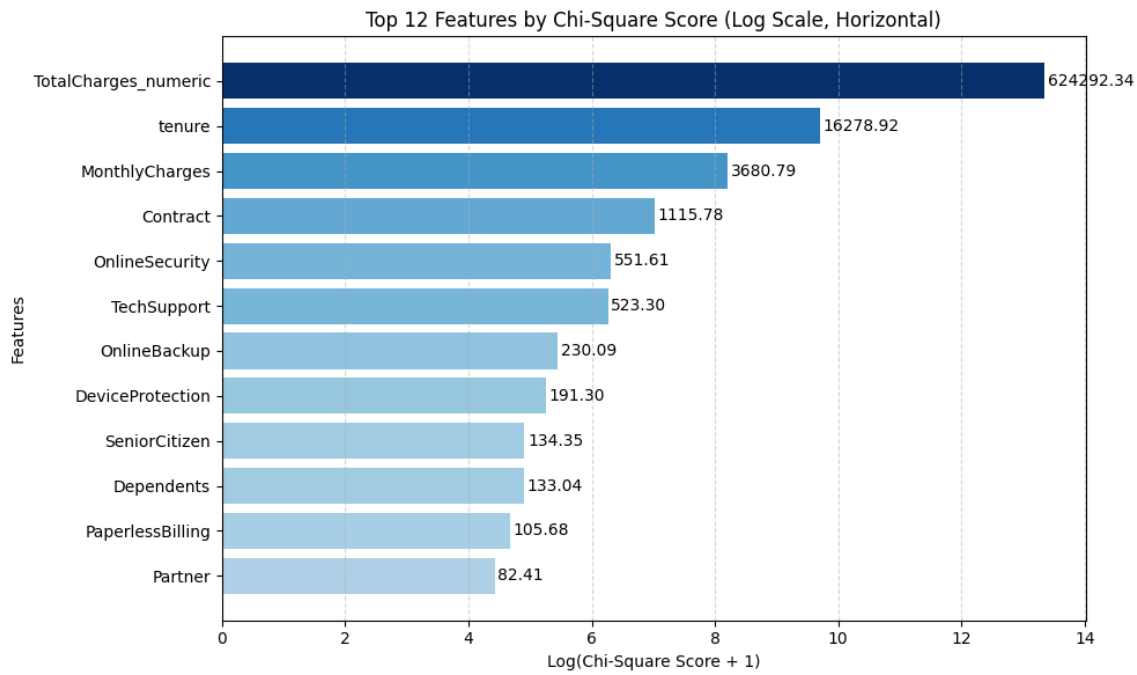

Figure 1 - Feature Selection (Neural Networks)

This method removed redundant or weak predictors and improved computational efficiency.

4.One-Hot Encoding

After selecting the top 12 categorical features, One-Hot Encoding was applied. This step was essential because:

- Neural Networks cannot work with categorical features.
- One-Hot Encoding ensures features are represented independently and avoids bias.

5. Outlier Analysis

Outlier detection using the IQR method was performed for numerical columns: No outliers found.

6. Skewness Handling

Skewness was inspected in numerical columns:

TotalCharges_numeric	0.963789
MonthlyCharges	-0.22524
Tenure	0.239540

Based on the range (-0.5 to +0.5) log transformation was applied to TotalCharges_numeric. This ensures that the neural network receives more normalized feature distributions.

7. Feature Scaling

All numeric attributes were rescaled with Min-Max normalization.

- Neural Networks perform best when inputs are within a small, consistent range (0-1).
- Min-Max scaling preserves the shape of the original distribution and not like standardization which shifts it.
- Works well with skewed features even after transformation.

8. Data Splitting

The data was partitioned into training and testing sets (80:20), with stratification to retain the original distribution of classes.
Training data: 5634
Testing data: 1409

9. Saving the Preprocessed Dataset and Split Values

The fully preprocessed dataset and the X Y splits were saved in Google drive so other Colab files can import them when needed.

10. Handling data imbalance

Class imbalance can cause the neural network to favor the majority class, negatively affecting performance metrics for the minority class

To address this Comparative Analysis of Imbalance Handling was done to find the best technique for this dataset.

Technique	Description
Class Weights only	Weighted loss function to penalize misclassification of minority class
SMOTE	Synthetic Minority Oversampling Technique
Borderline SMOTE-1	SMOTE focused on borderline minority instances
Borderline SMOTE-2	Variant of Borderline SMOTE with more conservative sampling
ADASYN	Adaptive synthetic sampling technique
SMOTE + Class Weights	Combination of oversampling and class weighting
Borderline SMOTE-1 + Class Weights	A hybrid approach of Borderline SMOTE-1 and class weights.
Borderline SMOTE-2 + Class Weights	A hybrid approach of Borderline SMOTE-2 and class weights.
ADASYN + Class Weights	Combination of ADASYN and class weights

Metric Comparison

Technique	Class_Weights	SMOTE	Borderline_SMOTE_1	Borderline_SMOTE_2	ADASYN	SMOTE_Class_Weights	Borderline_SMOTE_1_ Class_Weights	Borderline_SMOTE_2_ Class_Weights	ADASYN_Class_Weights
Train Accuracy	0.7808	0.7724	0.7393	0.7288	0.7364	0.7704	0.7498	0.7188	0.7446
Train Precision	0.5655	0.7982	0.782	0.7605	0.7832	0.7884	0.7593	0.775	0.77
Train Recall	0.7505	0.7292	0.6637	0.668	0.6508	0.7391	0.7316	0.6166	0.6947
Train F1 Score	0.645	0.7621	0.718	0.7113	0.7109	0.7629	0.7452	0.6868	0.7304
Test Accuracy	0.7665	0.7715	0.7779	0.7559	0.7743	0.7608	0.7495	0.7771	0.7566
Test Precision	0.544	0.5565	0.5656	0.5294	0.5619	0.5377	0.52	0.5667	0.5318
Test Recall	0.7433	0.6845	0.7032	0.7219	0.6791	0.7059	0.7299	0.6818	0.6925
Test F1 Score	0.6282	0.6139	0.6269	0.6109	0.615	0.6104	0.6073	0.6189	0.6016
Train Precision (Class 0	0.8978	0.7507	0.7079	0.704	0.7033	0.7544	0.741	0.6816	0.7238
Train Recall (Class 0)	0.7917	0.8157	0.8149	0.7896	0.8212	0.8016	0.7681	0.821	0.7942
Train F1 (Class 0)	0.8414	0.7818	0.7576	0.7443	0.7577	0.7773	0.7543	0.7448	0.7574
Train Precision (Class 1	0.5655	0.7982	0.782	0.7605	0.7832	0.7884	0.7593	0.775	0.77
Train Recall (Class 1)	0.7505	0.7292	0.6637	0.668	0.6508	0.7391	0.7316	0.6166	0.6947
Train F1 (Class 1)	0.645	0.7621	0.718	0.7113	0.7109	0.7629	0.7452	0.6868	0.7304
Test Precision (Class 0)	0.8931	0.8757	0.8824	0.8843	0.8746	0.8802	0.8857	0.8759	0.8753
Test Recall (Class 0)	0.7749	0.8029	0.8048	0.7681	0.8087	0.7807	0.7565	0.8116	0.7797
Test F1 (Class 0)	0.8298	0.8377	0.8418	0.8221	0.8404	0.8274	0.8161	0.8425	0.8247
Test Precision (Class 1)	0.544	0.5565	0.5656	0.5294	0.5619	0.5377	0.52	0.5667	0.5318
Test Recall (Class 1)	0.7433	0.6845	0.7032	0.7219	0.6791	0.7059	0.7299	0.6818	0.6925
Test F1 (Class 1)	0.6282	0.6139	0.6269	0.6109	0.615	0.6104	0.6073	0.6189	0.6016
Test CM TN	802	831	833	795	837	808	783	840	807
Test CM FP	233	204	202	240	198	227	252	195	228
Test CM FN	96	118	111	104	120	110	101	119	115
Test CM TP	278	256	263	270	254	264	273	255	259

Figure 2 - Comparative Analysis of Imbalance Handling Summary Chart

Among all the Imbalance handling techniques class weights shows better performance. Train accuracy for Class weights (0.7808) is the highest than others. It has a test accuracy of 0.7665 which shows that class weights provide a good balance between train/test performance without overfitting.

As this is an imbalance dataset accuracy can be misleading so Test F1 score was also considered among all the models Class Weights achieved the highest F1 score (0.6282) while also maintaining a higher recall (0.7433) for churn class. The precision is moderate but adhering to our assumption recall should be higher.

Confusion Matrix values are good comparing to the other model. True Positives is higher and False Negatives are lower which is exactly what we need for a business-oriented aspect.

Comparing to other techniques class weights are simple and stable and also we can preserve the model without creating unrealistic synthetic sample. Therefore, given that the system has business impact, model with Class Weights is the best performing model.

Model Building

1. Handling Class Imbalance (Training Set Only):

Based on the comparative analysis class weights were applied to minority class. Compute_class_weight (class_weight='balanced', ...) computes the class weights and pays more attention to the churn class.

2. Model Initialization

Used Sequential API from Keras to define the model as a linear stack of layers.

3. Model Architecture

- Input Layer + First Hidden Layer:
  - 32 units, ReLU activation, L2 regularization (λ=0.002)
  - Dropout 0.2
- Second Hidden Layer:
  - 16 units, ReLU activation, L2 regularization (λ=0.002)
  - Dropout 0.2
- Output Layer:
- Single neuron, Sigmoid activation

The neural network was built using a Sequential model with layers of size 32 → 16 → 1. The first hidden layer with 32 neurons captures complex feature interactions, while the second hidden layer with 16 neurons focuses on the most relevant patterns and reduces overfitting. The output layer has 1 neuron with a sigmoid activation for binary classification. ReLU helps the hidden layers learn better by adding flexibility and preventing common training problems. Dropout (0.2) was applied to prevent overfitting by randomly deactivating neurons, and L2 regularization (0.002) penalizes large weights, encouraging simpler and more robust patterns. This approach helps the model deal with complexity and generalize to new data successfully.

Model Compilation

- Optimizer: Adam
- Learning Rate: 0.0005
- Loss Function: Binary Cross-Entropy
- Metrics: Accuracy

This optimizer was chosen because it adjusts the learning rate automatically and handles rare updates efficiently, which is suitable for this neural network. A learning rate of 0.0005 allows steady convergence without overshooting the minimum. Binary cross-entropy is appropriate for the binary target variable. Accuracy is included as a standard metric.

Model Training

- Training Data: X_train, y_train
- Validation Data: X_val, y_val
- Batch Size: 32
- Epochs: 100
- Class Weights
- Callbacks:
  - EarlyStopping

- ReduceLROnPlateau

The model was trained using a batch size of 32. 100 epochs were chosen to give sufficient training time while EarlyStopping monitors validation loss to prevent overfitting. The validation data is used to keep track of how accurately the model performs on unseen data. and ReduceLROnPlateau adjusts learning rate dynamically if validation loss plateaus, improving convergence.

Model Evaluation

Threshold Tuning

Final Best-Chosen Threshold : 0.5299
To find the best cutoff point for turning predicted probabilities into class labels, the model's predictions were tested across thresholds ranging from 0.1 to 0.95, increasing by 0.01 each time. At each threshold, the F1-score was computed, and the threshold that gave the highest F1-score was chosen as the optimal one. This value provided the most balance between precision and recall for predicting churn.

Evaluation Metrics

	Training	Validation	Testing
Accuracy	0.7714	0.7835	0.7559
Precision	0.5518	0.5696	0.5284
Recall	0.7378	0.7525	0.7460
F1 Score	0.6314	0.6484	0.6186
Loss	0.4928	0.4894	0.5041
ROC-AUC Score	0.8439	0.8445	0.8369

The losses during training and validation are very close which tells us that there no severe overfitting. Accuracy is higher for Validation which depicts the model generalizes well. There is a higher recall which helps the model catch more churners. F1 is average due to the compromise between precision and recall. ROC-AUC values are high meaning the model can separate classes effectively.

Test loss is slightly higher it's may be due to model focusing on recall. Accuracy is lower than validation because the precision is low which pulls down overall accuracy. Test precision is low; this is also due to model prioritizing recall than precision (based on our assumption). Recall is considerably high here because missing a churner is costlier than falsely predicting one. Test F1 score is a trade-off so its moderate.

Overall, the model is optimized to detect churners at the cost of some false positives, which explains why accuracy and F1 score are moderate.

Classification Report - Test Set

Class	Precision	Recall	F1 Score	Support
0 (No churn)	0.89	0.76	0.82	1035
1 (Churn)	0.53	0.75	0.62	374
Macro Avg	0.71	0.75	0.72	1409
Weighted Avg	0.80	0.76	0.77	1409

The model performs better on the dominant class than smaller class. Recall is similar for both classes, meaning that the model is trained to capture all churners while also misclassifying non churners as churners. This result is due to the imbalance in the test set. Overall, the weighted averages (F1 = 0.77) reflect the model's balanced attention to both classes.

Confusion Matrix

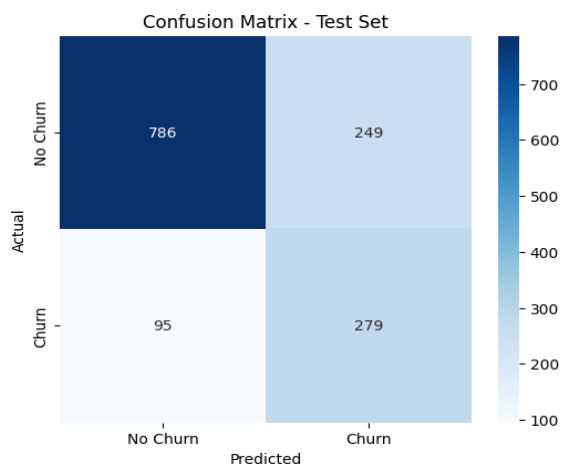

Figure 3 - Confusion Matrix (NN)

- True Positives (279): Correctly predicted churners
- False Negatives (95): Missed churners
- False Positives (249): Predicted churn but actually non-churn
- True Negatives (786): Correctly predicted non-churn

ROC Curve

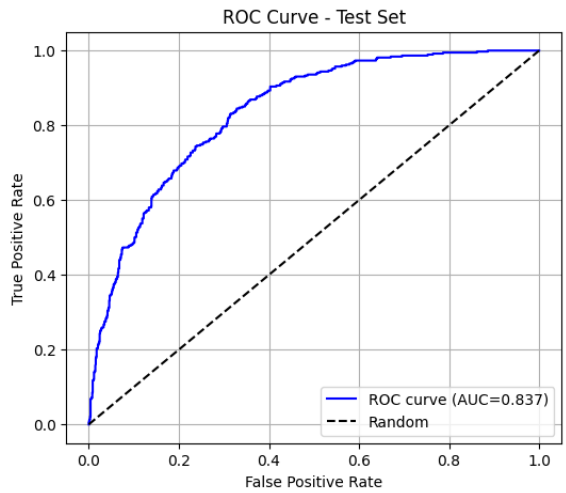

Figure 4 - ROC Curve (NN)

With an AUC score of 0.837, the model demonstrates solid classification ability. In practical terms, it correctly ranks a positive example above a negative one about 84% of the time.

The curve sits significantly above the random line across the entire plot, confirming that the model provides substantial predictive value.

Precision-Recall Curve

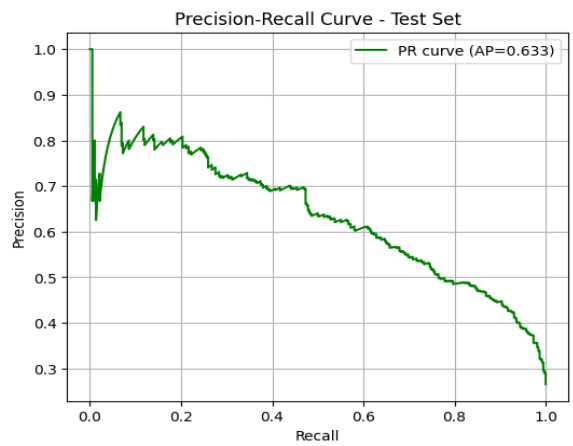

Figure 5 - Precision Recall Curve (NN)

Average Precision = 0.633. This confirms that the model is effective for the minority class.

The curve shows the model can retrieve about more than half of the actual churners while maintaining a prediction accuracy of over 60%.

Training vs Validation Loss

The Training vs Validation Loss graph shows that both losses decrease rapidly in the initial epochs, with the training loss (blue) stabilizing little bit lower than the validation loss (red). After epoch 5 the model slowly converges and the losses start to flatten out. By epoch 30, the loss is stable around 0.52 and the val_loss is also relatively stable, fluctuating between 0.50 and 0.52. The most striking observation is that Training loss exceeds validation loss, this is due to class weights as training data is penalized for losses while validation data is not applied with class weights. Usage of regularization also increases loss during training. Overall, the losses converge and stabilize, suggesting that the model has effectively learned the underlying patterns without severe overfitting.

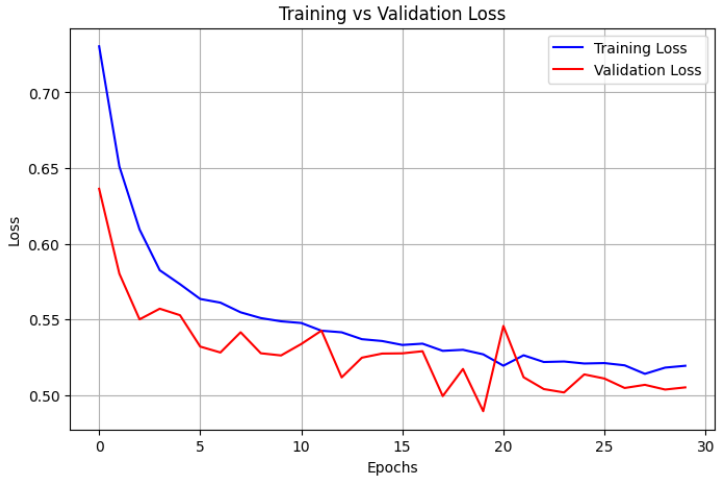

Figure 6 - Training Loss vs Loss Graph

Baseline vs. Final Model Comparison

To evaluate the impact of the improvements applied in model architecture tuning, the performance of the final neural network model was benchmarked against the baseline model. The baseline model was trained without balancing classes, without threshold tuning, and with a simpler training strategy.

Metric	Baseline Model	Final Model	Improvement
Accuracy	0.7970	0.7559	Slightly lower (expected due to higher recall focus)
Precision	0.6286	0.5284	Lower (due to more churn predictions)
Recall	0.5748	0.7460	Significant improvement
F1 Score	0.6005	0.6186	Improved

Interpretation

- The baseline model prioritized precision and overall accuracy but missed many actual churners, leading to a lower recall whereas the final model was deliberately optimized to improve recall.
- A slight drop in accuracy and precision, which is acceptable because recall is more important for churn prediction (missing a churner is costlier than contacting a non-churner).

Confusion Matrix Comparison

Actual/Predicted	Baseline: Pred 0	Final: Pred 0	Baseline: Pred 1	Final: Pred 1

Actual 0	908	786	127	249
Actual 1	159	95	215	279

FN (missed churners): 95 → Reduced by more than half
This is the clearest indicator that the final model is far better at capturing churners, which is the main business objective.

Conclusion of Comparison (Baseline Vs Final)

The final neural network represents a substantial improvement over the baseline, particularly in identifying churners (Recall: 0.57 → 0.74). Despite a small trade-off in precision and accuracy, the increase in recall and F1-score makes it significantly more effective for real-world churn prediction, where failing to detect churners leads to financial loss.

2.1.7. Inference

Using the model, churn was forecasted for 10 customers from the test dataset. The table summarizes the predictions with the actual outcomes. Out of the 10 inference samples, 9 predictions were correct, indicating the model performs well on the inference set.

#	Actual	Predicted	Correct?
0	0	0	Yes
1	0	0	Yes
2	0	0	Yes
3	0	1	No
4	0	0	Yes
5	1	0	Yes
6	1	0	Yes
7	1	0	Yes
8	1	0	Yes
9	1	0	Yes